

Systematic literature review on the impacts of agile release engineering practices



Teemu Karvonen*, Woubshet Behutiye, Markku Oivo, Pasi Kuvaja

University of Oulu, Pentti Kaiteran Katu 1, 90014, Finland

ARTICLE INFO

Article history:

Received 23 May 2016

Revised 12 January 2017

Accepted 24 January 2017

Available online 25 January 2017

Keywords:

Release engineering

Agile

Continuous integration

Rapid release

Continuous delivery

Continuous deployment

ABSTRACT

Context: Agile release engineering (ARE) practices are designed to deliver software faster and cheaper to end users; hence, claims of such impacts should be validated by rigorous and relevant empirical studies.

Objective: The study objective was to analyze both direct and indirect impacts of ARE practices as well as to determine how they have been empirically studied.

Method: The study applied the systematic literature review research method. ARE practices were identified in empirical studies by searching articles for “rapid release,” “continuous integration,” “continuous delivery,” and “continuous deployment.” We systematically analyzed 619 articles and selected 71 primary studies for deeper investigation. The impacts of ARE practices were analyzed from three viewpoints: impacts associated with adoption of the practice, prevalence of the practice, and success of software development.

Results: The results indicated that ARE practices can create shorter lead times and better communication within and between development teams. However, challenges and drawbacks were also found in change management, software quality assurance, and stakeholder acceptance. The analysis revealed that 33 out of 71 primary studies were casual experience reports that had neither an explicit research method nor a data collection approach specified, and 23 out of 38 empirical studies applied qualitative methods, such as interviews, among practitioners. Additionally, 12 studies applied quantitative methods, such as mining of software repositories. Only three empirical studies combined these research approaches.

Conclusion: ARE practices can contribute to improved efficiency of the development process. Moreover, release stakeholders can develop a better understanding of the software project's status. Future empirical studies should consider the comprehensive reporting of the context and how the practice is implemented instead of merely referring to usage of the practice. In addition, different stakeholder points of view, such as customer perceptions regarding ARE practices, still clearly require further research.

© 2017 Elsevier B.V. All rights reserved.

1. Introduction

In software development, fast, incremental deliveries involve lightweight, efficient practices for continuous release planning [1] and release engineering [2,3]. This paper focuses on engineering by synthesizing empirical studies for *agile release engineering* (ARE) practices. To the best of our knowledge, the concept of ARE has not been used before in other scientific papers. We use it in this paper to map the research topic and to incorporate the investigated release engineering practices that are involved in a “*release engineering pipeline*” [3]. Hence, by ARE practices, we mean

contemporary software *integration, testing, deployment and release* practices that are often applied in modern release engineering [3]. Many of these practices originate from agile software development methodologies such as extreme programming (XP) [4]. From the research point of view, ARE practices derive from the theories of *agile and lean software development* [5], *release engineering* [2,3,6] and *continuous software engineering* [7,8] research disciplines. According to Adams et al. [6], release engineering “deals with all activities in between regular development and delivery of a software product to the end user, i.e., integration, build, test execution, packaging and delivery of software.” Continuous software engineering is an emerging subtopic in software engineering (SE) that is focused on continuous experimentation, innovation and the elimination of discontinuities within and between the developmental, operational and business strategy functions. Fitzgerald and Stol [8] associate concepts in continuous software engineering with concepts of classic “lean thinking” [9] such as “value and waste,”

* Corresponding author.

E-mail addresses: teemu.3.karvonen@oulu.fi (T. Karvonen), woubshet.behutiye@oulu.fi (W. Behutiye), markku.oivo@oulu.fi (M. Oivo), pasi.kuvaja@oulu.fi (P. Kuvaja).

“flow and batch size,” “autonomation and building-in quality” and “Kaizen and continuous improvement.”

Modern ARE practices are supposed to aid in delivering software faster and cheaper to end users; hence, claims of such impacts should be validated by rigorous and relevant empirical studies. In this systematic review, the objective is to understand the direct and indirect impacts of ARE practices. By investigating the direct and indirect impacts we emphasize the notion that “impacts may be desired already according to the explicit method rationale(s), or they may be unexpected, sometimes even unwanted[10].” In addition, we sought to evaluate primary studies to understand how the impacts of ARE practices have been investigated in empirical studies. The research questions are:

RQ1: What are the direct and indirect impacts of ARE practices? We break down the main question into three sub-questions as follows:

RQ1a: What are the impacts associated with adoption of ARE practices?

RQ1b: What is the prevalence of ARE practices?

RQ1c: What are the impacts of ARE practices on the success of SW development?

In addition, we define a second main research question to understand how these impacts have been investigated:

RQ2: How have ARE practices been investigated in empirical studies?

In this study, we systematically searched and analyzed empirical studies investigating 1) *continuous integration* (CI) [11,12], 2) *continuous delivery* (CD) [12], 3) *rapid release* (RR) [13] and 4) *continuous deployment* (CD2) [11,12]. We analyzed and clustered studies by topic and research approach, and outlined a checklist for analyzing software development capabilities for CD2 in the context of software-intensive products. We applied a systematic literature review (SLR) [14] method that allowed us to critically compare, evaluate and synthesize the primary studies. Our main selection criterion for the primary studies was that they were conducted in real software development contexts. Literature reviews, mapping studies, opinion papers and small-scale experiments with students are not included in our analysis. To the best of our knowledge, a systematic synthesis focusing on the impacts of ARE practices has not previously been undertaken, although some of the practices have been synthesized either separately or from a different research question point of view, as we explain in more detail in the following section. With this systematic review, we aim to provide a reliable overview of the current state of existing empirical studies for ARE practices that may help in terms of scoping and planning future studies. Our study also helps practitioners to better understand the impacts and capabilities associated with ARE practices. Finally, this paper aims to contribute to the theorizing on software development practices [10] for ARE. The concepts used in this paper (i.e., *learning, practice, development context, rationale, impact* and *theory*) conform to definitions used for the Coat Hanger model [10].

2. Background

ARE practices aim to support the agile principle of “early and continuous delivery of valuable software [15].” Early and continuous deliveries allow mechanisms for fast feedback and transparency of the development process, allowing stakeholders to continuously review and evaluate the state of the system under development and, if needed, to make adjustments to the priority and content requirements accordingly. CI practice originated from the agile XP methodology. Beck and Andres [4] summarizes CI as

follows: “New code is integrated with the current system after no more than a few hours. When integrating, the whole system is built from scratch and all tests must pass or the changes are discarded.” CI is often characterized by development conventions and tools for the automation of the build and test activities [11]; XP, however, goes beyond tools and emphasizes *values* and *principles* [4] that *rationalize* [10] the usage of CI practice. Development team members must prioritize development activities to “commit changes frequently” and “fix broken builds immediately [4].” Consequently, “all tests and inspections must pass” (i.e., with zero tolerance for regression and failed test cases) [4].

It is safe to assume that the impacts of CI practice depend on the actual implementation of the practice as well as the stage of assimilating the practice the organization is in. In their literature review, Eck et al. [16] investigated CI practice from the assimilation point of view. They identified 14 distinct organizational implications of CI in three assimilation stages:

1. Acceptance: devising an assimilation path, overcoming the initial learning phase, dealing with test failures right away and introducing CI for complex systems.
2. Routinization: institutionalizing CI, clarifying the division of labor, CI and distributed development, mastering test-driven development and providing CI at project start.
3. Infusion: CI assimilation metrics, devising a branching strategy, decreasing test result latency, fostering customer involvement in testing and extending CI beyond the source code.

Another related literature review by Ståhl and Bosch [17] focused on technical aspects of the implementation of CI practice. Moreover, they analyzed variations in the interpretation and implementation of CI practice. They concluded that CI practice interpretations and implementations are different from case to case. For example, integration and test flow frequency vary in different contexts. Meanwhile, Päiväranta and Smolander [10] state that predefined practices or methods in theory need to be distinguished from the contextual practice descriptions of what actually happens. Subsequently identified CI impacts, as well as other ARE practices in our systematic review, may not apply to practice in theory but rather to different variants (context-specific embodiments) of the practice, which may evolve dynamically over time. Ståhl and Bosch [17] also state that “in order to make a meaningful comparison of software development projects, simply stating that they use continuous integration is insufficient information as we instead need to ask ourselves what kind of continuous integration is used.” To better understand CI practice variations, Ståhl and Bosch introduced a descriptive model that allows for the visualization of CI flow for a better understanding of context-specific variation points. Nevertheless, understanding technical implementations of CI practice alone may still not provide enough information to understand the causalities between CI practice and its impacts, as many other variables may be related to the adoption of its values and principles that may also either increase or diminish the impacts.

Within CI practice, multiple levels of integration may exist and changes may also be deployed in different kinds of environments. For example, the deployment environment could be a purely experimental test environment, a production-like environment or the actual production environment. CD practice is often considered to extend CI practice. Humble and Farley [12] introduce CD as a practice to automate the software delivery process for production purposes. CD promotes the idea of delivering software “at will,” i.e., the delivery can occur at any point in time with very little manual labor required. CD is often used interchangeably or as a synonym for CD2 practice; however, there are also accounts regarding how these practices differ. According to Fitzgerald and Stol [8], “continuous delivery is a prerequisite for continuous deployment, but

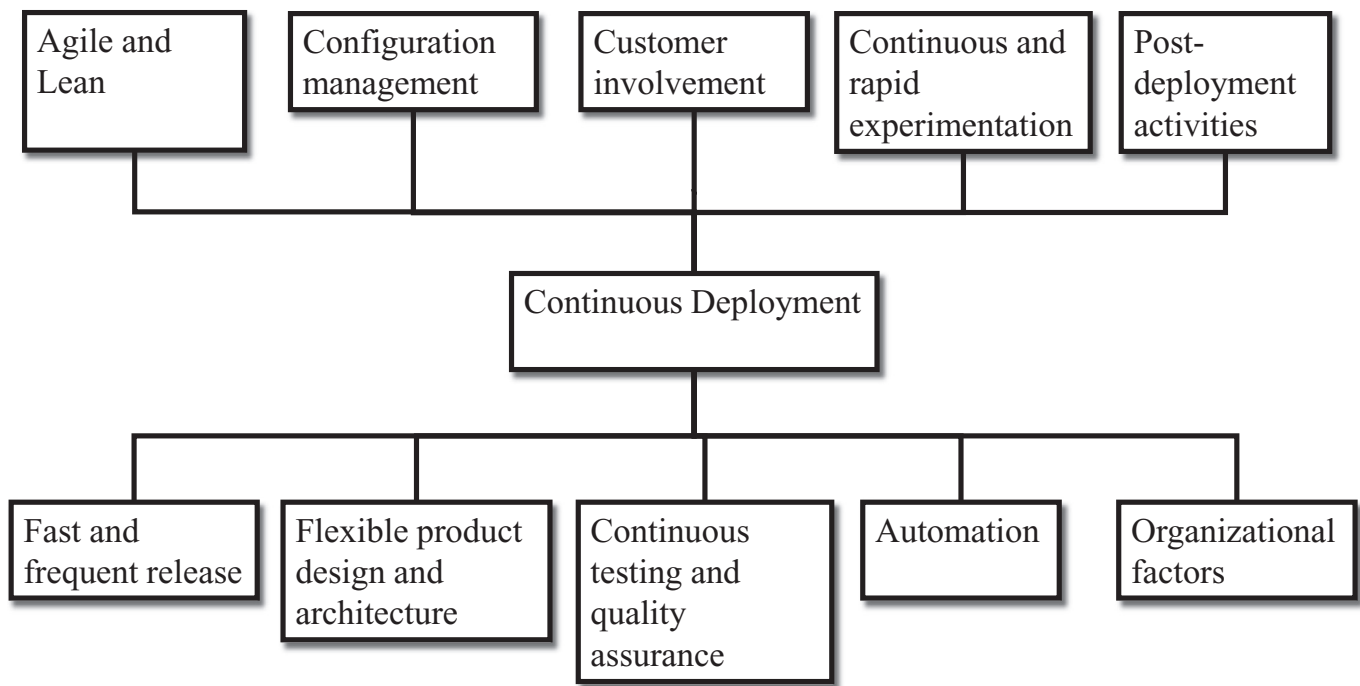


Fig. 1. Factors of continuous deployment. Rodríguez et al. [19].

the reverse is not necessarily the case. That is, continuous deployment refers to releasing valid software builds to users automatically, whereas CD refers to the ability to deploy the software to some environment, but not automatically deploy it to customers.” According to Humble [18], “deployment does not imply release” because deployment could be made for test- or production-like environments. Humble [18] continues, “continuous deployment is the practice of releasing every good build to users—a more accurate name might have been continuous release ... Continuous delivery is about putting the release schedule in the hands of the business, not in the hands of IT.”

A comprehensive mapping study [19] investigated CD2 factors in software-intensive product development. They identified ten recurrent themes from the literature involved in CD2 (Fig. 1). Specifically, factors such as *customer involvement*, *continuous rapid experimentation* and *post-deployment activities* clearly extend the scope of CI practice. Consequently, referring to CD2 as a development practice could belittle the magnitude of activities that are involved in CD2 practice. It may be justified to consider CD2 as a development and business planning *paradigm* that goes beyond agile software development.

The CI and CD2 concepts are also used in the context of the *stairway to heaven* (STH) model [20]. The STH model describes them as *preceding evolutionary steps* towards *innovation experiment systems* (IES) [21]. IES refers to the research and development (R&D) capability for experiment-driven development that could supersede the traditional periodic approach to requirement planning and prioritization. Bosch [21] highlights three key aspects in which the IES approach is different from the traditional approaches to developing software: “First, it is focused on continuously evolving the software by frequently deploying new versions. Second, customer and customer usage data play a central role throughout the development process. Third, development is focused on innovation and testing as many ideas as possible with customers to drive customer satisfaction and, consequently, revenue growths.” More recently, the practices associated with the STH model’s evolutionary CI and CD2 steps have been discussed in conjunction with several case studies [20,22–26], while applying the STH model as a frame-

work for describing the position and maturity of software companies in relation to their IES capability. Karvonen et al. [22] also extend the STH model by presenting an approach for analyzing software companies’ practices in relation to the business, architecture, process and organizational dimensions.

The RR concept has recently been used especially in the context of the Chrome [27] and Firefox [28] web browser release strategies. There are also empirical studies [29,30] using metrics derived from the Firefox development metrics as primary data for analyzing the impacts of a transition to shorter release cycles. Both the Chrome and Firefox web browsers use components developed in open-source software projects and their current releases are scheduled every six weeks. Earlier main versions were released at much longer intervals, from 12–18 months. Consequently, the development and release engineering practices had to be adjusted to support shorter release cycles. Mäntylä et al.’s [13] semi-SLR elucidates the RR concept further, as they originate RR in agile methodologies, the free and open-source software community, lean software development and “Internet time” high-speed software development.

3. Research design and implementation

We applied an SLR [14,31] method in our study. The research was conducted in seven key phases: 1) Research planning; 2) Piloting of the search; 3) Searching articles; 4) Inclusion and exclusion of articles; 5) Quality assessment of primary studies; 6) Analysis and clustering of primary studies; and 7) Reporting the results of the SLR. The first version of the research plan and protocol was written and reviewed in a research group comprising all the authors. Later, the plan and protocol were updated to scope the research question and add a more detailed description of each phase. After the initial version of the research protocol was written and reviewed in the research group, we moved on to piloting the search phase. Various versions of the search strings were tested, mainly in the Scopus and IEEE Xplore databases. Search string A (presented in Table 1) was considered to result in an adequate number of papers to complete the SLR. However, during the inclusion and exclusion steps, we learned that one important

Table 1
Search strings used in the SLR.

Search strings	Search date
Search string A: ((software AND ('continuous deployment' OR 'continuous delivery' OR 'continuous integration')).	29.6.2015
Search string B: (software AND 'rapid releas*')	1.8.2015

search term (“rapid release”) was missing. Therefore, we moved back to the search phase and complemented the search with the second search string, B (Table 1). In our review, we considered RR to be a valuable search term for analyzing ARE practices as “rapid” in conjunction with “release” which could provide relevant empirical studies for analyzing the impacts of ARE practices. Each step of the study selection process and the study quality assessment process was duplicated by having two authors work individually at first, documenting their decisions in Excel sheets, and then peer-reviewing their inclusion, exclusion and assessment decisions. Peer-reviewing included a comparison of decisions and a conflict-resolution process achieved through discussions regarding the content of each paper and the criteria for inclusion, exclusion and quality. Throughout the process, we considered the rate of conflicting decisions to be rather low. Our analysis showed that 6% (37 from 619) of the articles had conflicting decisions when comparing results after completing STEP4 (exclude articles based on introduction). In addition, the quality of the primary studies was also evaluated by the same two authors following a similar conflict-resolution procedure.

We searched articles from six databases: Scopus, IEEE Xplore, ACM Digital Library, ISI Web of Science, Science Direct and Springer Link. All searches were made for the article title, abstract and keyword fields, except for Springer Link, where the search also included full text. This was due to Springer Link search functionality that did not provide an option for specifying article sections to be searched. These databases were considered to contain studies for most of the relevant journals and conference proceedings in the software engineering discipline. We learned during the search phase that Scopus alone includes most of the abstracts and citations to articles found in other databases. However, the most recent conference proceedings, such as for the International Conference on Agile Software Development (XP) 2015, had not yet been imported into Scopus. Therefore, we decided to search the Springer Link database for articles published in 2015.

The steps taken during the inclusion and exclusion phase were as follows: STEP1: Exclude duplicate articles; STEP2: Exclude articles based on title; STEP3: Exclude articles based on abstract; STEP4: Exclude articles based on introduction; and STEP5: Exclude articles based on reading the full paper. Table 2 shows the statistics from the inclusion and exclusion steps. In each step, we excluded papers that clearly were not in the scope of ARE practices. We excluded all literature reviews and articles that were not conducted in a real software development context, e.g., studies using students as informants. In STEP3 and STEP4, we also excluded some papers that focused on the construction or evaluation of tools that could be attached to CI systems. The reasoning behind this was that those papers mostly focused on the evaluation of a tool and not on the impacts of ARE practices. During the selection steps, the number of primary studies was narrowed down from 619 to 71. These papers were moved to the assessment and analysis phase. The results are presented in the following section.

Typically, a paper might be excluded after being fully read if its main topic turned out to be something other than ARE practices (ARE practices were merely mentioned in the title, abstract or introduction). In addition, some articles were excluded as they did not include any details about the context. Those articles were con-

sidered to be opinion papers, i.e., the context was not presented and the claims of the paper could not be traced to any software development project, product, service or organization.

4. Clustering and assessment of the primary studies

This section presents the results of the assessment on the rigor of the primary study and its relevance to the industry. In addition, we cluster articles based on their research approach and main topics in terms of providing an answer to RQ2 (How have ARE practices been investigated in empirical studies?).

After undertaking the inclusion steps presented in the previous section, the articles were read multiple times and analyzed using the NVivo tool [32] to extract data for the synthesis and clustering of the primary studies. In addition to using NVivo as the main repository for storing PDF versions of the articles, NVivo's node (qualitative data coding), classification, linking, and memo functionalities were used for conducting the assessment and analysis steps, i.e., each included primary study was assigned attributes such as study topic, research approach, context etc. Corresponding article sections referring to the impacts of ARE practices were coded using NVivo's node functionality. Finally, the memo functionality was used to extract data from articles to text format, which could be used to synthesize the results for this paper. The final clustering based on article attributes was done in collaboration with the authors. A research topic could often be determined from article title, abstract or keywords, since authors typically position their study clearly by a certain topic, such as CI. However, sometimes determining of the topic required a more comprehensive analysis of the full paper.

For the assessment, we applied Ivarsson and Gorschek's scale [33] for analyzing the study's rigor and its industrial relevance. The scale is commonly used in various systematic mappings and reviews in software engineering, although it was originally designed for technology evaluations. The scale applies three aspects to evaluate study rigor (*context described*, *study design described*, and *validity discussed*) with scale levels of 0 (*strong description*), 0.5 (*medium description*), and 1 (*weak description*). The scale also has four aspects to evaluate study relevance (*subjects*, *context*, *scale*, and *research method*) with scale levels of 0 (*do not contribute for relevance*) and 1 (*contribute to relevance*).

The results of the assessment articles were clustered into two categories, entitled *experience reports* (33 articles) and *empirical studies* (38 articles). To name these categories, we applied terms commonly used in conference calls to discuss both casual experience and more formal scientific contribution. The articles categorized as empirical studies had at least a medium-level (a score equal to or greater than 0.5) presentation of the context and study design. In 12 (31.6%) out of 38 empirical studies, the validity discussion was considered to be weak (score = 0). The experience reports were considered to be relevant since they presented the research method's “lessons learned” in a context that was clearly described in the article, i.e., the context was described at least at a medium level (a score equal to or greater than 0.5). The articles that did not describe the context at all (score = 0) were considered to be opinion papers and they were excluded from further analysis. Although experience reports could provide interesting insights into ARE practices, in contrast to the empirical studies, the experience reports do not describe an explicit research question, study design or include a validity threat discussion. For these reasons, we did not evaluate the rigor aspect for the study design or the validity discussion in the experience reports (score = not applicable). Consequently, the claims about the impacts of ARE practices that were presented in the experience reports are unreliable, i.e., the experience reports are considered not to be as reliable a source of information as the empirical studies are. Detailed results of the

Table 2
Inclusion and exclusion of primary studies.

Search string	Papers found	STEP1	STEP2	STEP3	STEP4	STEP5	Included for analysis
A	546	110	120	210	41	6	59
B	73	12	11	30	7	1	12
A + B	619	122	131	240	48	7	71

Table 3
Clustering of primary studies by main topic.

Topic:	Experience reports	#	Empirical studies	#
Agile methods & XP	–	0	[34–37]	4
Continuous deployment (CD2)	[38,39]	2	[20,22–25,40,41]	7
DevOps	[42,43]	2	[44,45]	2
Rapid release (RR)	–	0	[13,29,30,46–54]	12
Continuous delivery (CD)	[55–59]	5	[60–63]	4
Continuous integration (CI)	[64–87]	24	[88–95]	8
Innovation experiment systems (IES)	–	0	[26]	1
Total:		33	Total:	38

Table 4
Clustering of primary studies by research approach in relation to impacts of ARE practices.

A. Studies investigating the impacts associated with changing the development practice	References	Number of studies: 22
A.1: Advantages and challenges of applying the practice.	[20,22–26,34,35,40,41,44,62,88]	13
A.2: Strategies to mitigate challenges associated with the adoption of the practice.	[22,25,41,93]	4
A.3: Reconciling of existing software development processes with the practice.	[37,45,60,63,92]	5
B. Studies investigating the prevalence of the practice in software-intensive projects	References	Number of studies: 7
B.1: Prevalence of the practice in open-source software development.	[89]	1
B.2: Prevalence of the practice in the Finnish ICT industry.	[26,40]	2
B.3: Prevalence of the practice in GITHUB projects.	[95]	1
B.4: Prevalence of the practice in large distributed international companies.	[37]	1
B.5: Prevalence of the practice in the mobile application development domain.	[61]	1
B.6: Prevalence of the practice in Malaysia	[34]	1
C. Studies investigating the impact of the practice on the success of software development	References	Number of studies: 17
C.1: Impact on project status communication.	[90,93]	2
C.2: Impact on stakeholder satisfaction.	[91]	1
C.3: Impact on developer productivity and project predictability.	[93,94]	2
C.4: Impact on software security.	[46]	1
C.5: Impact on project lead time.	[47]	1
C.6: Impact on software errors.	[29,48]	2
C.7: Impact on software testing.	[13,30]	2
C.8: Impact on code review activity.	[49]	1
C.9: Impact on software integration delays.	[50]	1
C.10: Impact on manual test-case prioritization.	[51]	1
C.11: Impact on stabilization of software releases.	[52]	1
C.12: Impact on software integration rework (backout rate).	[53,54]	2
D. Lessons learned when implementing the practice	References	Number of studies: 33
D.1. Lessons learned when using ARE practices.	[38,39,42,43,55–59,64–87]	33

quality assessment can be found in the data sheet in Appendix A at <https://goo.gl/nhDGKE>. Table 3 presents the clustering of the 71 primary studies by their main topic.

Table 4 presents the clustering of the empirical studies based on their research approach. We were able to identify 17 studies that focused on investigating and quantifying the practice's impact on software development success factors such as software quality, stakeholder satisfaction, lead times and the number of error reports before and after adoption of the practices. The majority (22) of studies focused on investigating the impacts associated with the adoption of the practice, e.g., understanding and managing the change in the practice. In addition, some of these studies suggested strategies for mitigating challenges (unwanted impacts) when changing practice and how to reconcile existing development processes with the new practice. Seven studies investigated the prevalence of the practice in software-intensive projects, i.e., the impact regarding the popularity or significance of the practice on the software development domain.

Fig. 2 illustrates various approaches for investigating ARE practices, with examples of the research questions that could be used: 1) investigating/quantifying impacts associated with the adoption

of the practice; 2) impacts associated with the prevalence and significance of the practice in the software development domain; 3) investigating impacts associated with the success of software development; and 4) experience reports that provide insights into how the practice can be implemented. Table 5 presents the clustering of the primary studies based on their data-collection approach. After experience reports (33 papers), the most typical data-collection approach was qualitative interviews or surveys with practitioners (23 studies). We were able to identify 12 studies that used work products such as log files and error reports to analyze the impacts of ARE practices. Only three studies used both of these approaches for data triangulation. A large proportion, 26.3% (10 out of 38), of the empirical studies used data collected from the Mozilla Firefox development context. More details about the research context can be found in the data sheet in Appendix A at <https://goo.gl/nhDGKE>.

5. Analysis of the impacts of ARE practices

This section elaborates the key findings from the primary studies for providing the answer to RQ1 (What are the direct and in-

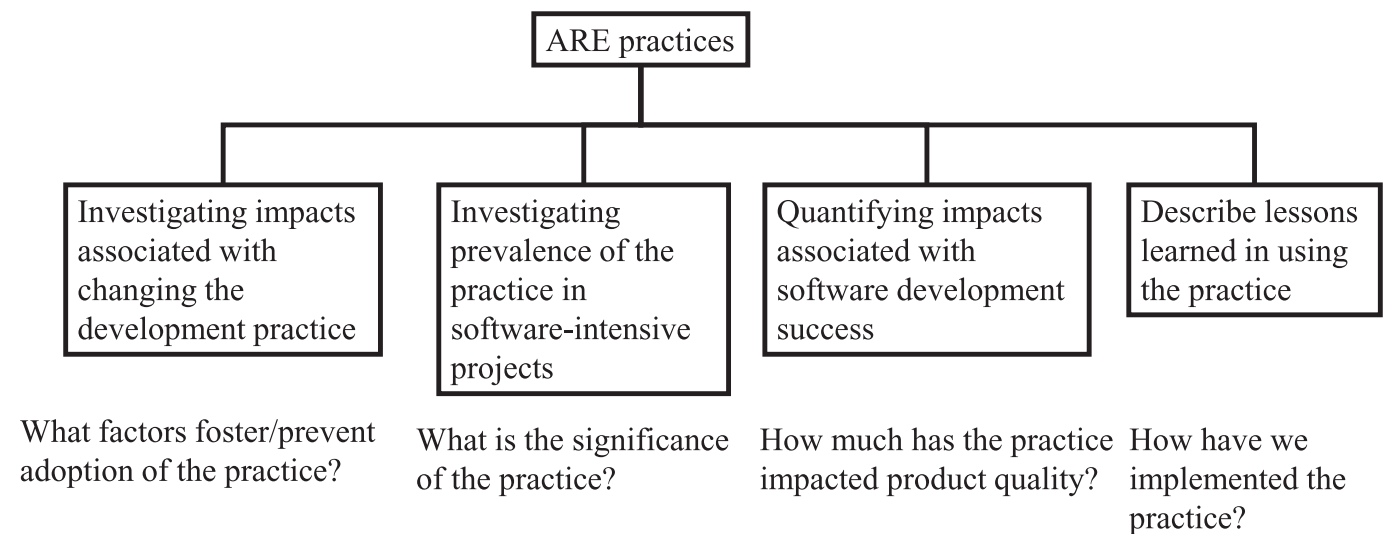


Fig. 2. Research approaches with examples of research questions.

Table 5
Clustering of primary studies based on the data collection approach.

Data-collection approach	References	Number of studies
A. Studies analyzing data collected via work products, error reports, log files, etc.	[29,46,48–54,61,89,95]	Number of studies: 12
B. Studies analyzing data collected via surveys or interviews with stakeholders	[20,22–26,34–37,40,41,44,45,47,60,62,63,88,90–92,94]	Number of studies: 23
C. Studies applying both the A and B approaches for data collection	[13,30,93]	Number of studies: 3
D. Experience reports with no data collection approach specified	[38,39,42,43,55–59,64–87]	Number of studies: 33

direct impacts of ARE practices?). To structure the results we have specified three sub-questions: RQ1a (What are the impacts associated with adoption of ARE practices?), RQ1b (What is the prevalence of ARE practices?) and RQ1c (What are the impacts of ARE practices on the success of SW development?).

In Section 5.1, we focus on adoption of the practice. This viewpoint involves the practitioner’s perceptions regarding challenges and benefits (i.e., positive and negative impacts) associated with adoption of ARE practices. We also synthesize identified key capabilities for ARE, strategies to mitigate the challenges associated with adoption of ARE practices, and methods for reconciling existing development practices with ARE.

In Section 5.2, we consider the prevalence of ARE practices, e.g., usage of the practice in software development projects, such as CI practice usage in different contexts. This viewpoint also addresses the bigger picture of how ARE impacts the work of software development professionals and researchers in the field.

Finally, in Section 5.3, we elaborate identified contextual impacts associated with software development success, such as software quality factors (i.e., error-proneness of ARE practices) and process efficiency (i.e., lead times in using ARE practice).

5.1. Analyzing impacts associated with adoption of ARE practices (RQ1a)

According to our analysis, investigation of the aspects associated with changing the development practice was the most common approach among the primary studies (Table 4, cluster A). Understanding the related challenges and intermediate steps in adoption of ARE in different contexts can allow for more accurate planning of strategies and activities for enhancing adoption and assimilation of the practice.

According to Debbiche et al. [88], several challenges can arise when adopting CI. They state that some of these challenges can be associated with *domain-specific constraints* such as *product com-*

plexity and *process suitability*. The limitations and challenges of using ARE practices in different business and industry domains have been identified, especially in the business-to-business (B2B) [62] and embedded system [22] domains. According to Debbiche et al. [88], the same integration, test and release frequency may not fit all organizational processes. Several software developers highlight the problem of using CI for all parts of the product and its features; the optimal CI flow frequency may depend on the type of work that is being carried out. Consequently, the optimal speed of integration and the optimal testing frequency may be limited for upstream and downstream process activities such as the capability of breaking down software requirements [88] and the internal or external customer capabilities in terms of receiving the software [88].

Debbiche et al.’s [88] study suggests that efficient CI flow requires *breaking down the requirements* into parts that can be prioritized and balanced regarding both size and testability. CI flow must also take into account the constant need for refactoring, as well as minor changes that may not explicitly add value to a feature, but are nonetheless worth integrating [88]. High-frequency CI testing and integration activities may also be hindered by product size, design complexity and a large number of product branches [88]. As identified by Debbiche et al. [88], CI flow efficiency may deteriorate due to unclear component interfaces, difficulties in locating the error source between multiple teams, integration failures and waiting for other components (parts) to be completed. Hence, technical solutions should be developed to integrate partial code for a feature, e.g., using code toggle “switches” to activate a feature for integration and testing when all the parts are ready [88]. In addition, to manage the risks associated with unsuccessful deployments, the product architecture should support mechanisms to deploy parts of the system and also to roll back unsuccessful deployments, as noted in several studies [22–25].

Bellomo et al. [60] investigated CD practice in relation to *deployability* improvement goals and architectural design decisions in

three projects from different organizations. Their analysis shows that the main deployability improvement goals are “build and continuous integration,” “test automation,” “deployment and robust operations” and “synchronized and flexible environments”. Consequently, they state that the transition to CD practice requires new *tactics for architectural design* such as “an integrated test framework,” “script-driven process shutdown,” “web service consolidation,” “parameterization,” “self-monitoring,” “adapter container,” “single-responsibility principle and distributed service architecture,” “managing and reproducing state,” “self-initiating version update,” “monitoring and auto-scaling,” “removing web services and collapsing the middle tier,” “load balancer,” and “bundle and rollback feature and data layer change” [60]. Architecture-related requirements and tactics for the transition to IES have also been addressed in other studies and contexts: support for A/B testing [22,23], modularity [22,23], instrumentation for data collection [23], testability [22], run-time functionality variation [22] and the independent deployability of functionalities [22].

The existing CI and CD literature [11,12] emphasizes various development tools and infrastructures (e.g., deployment pipeline). Consequently, although ARE tools are not within the scope of our study, the role of tools in adoption of ARE practices is central. Hence, an explicit analysis of ARE tools could be a relevant topic for future studies. CI flow may lack efficiency due to a lack of maturity, with an absence of appropriate tools and infrastructure for code reviews, integration queue management and the tracking of deliveries during large-scale developments, as stated by Debbiche et al. [88]. Subsequently, maintaining the quality of the product becomes difficult without sufficient infrastructure and tool support for CI flow test automation, as manual testing may not be scalable when more frequent integration cycles are required. Many defects and requirements surface only when the system is used in its intended context [92]. Therefore, automated and manual system-level testing activities and tools should provide fast feedback flows to requirement planning and product development, as identified by Ståhl and Bosch [93] and Knauss et al. [92]. Delayed regression test feedback may negatively impact the CI flow benefits and efficiency. Based on interviews held with practitioners, Ståhl and Bosch [94] report that CI practice can improve communication both within and between development teams. However, their interviewees gave divergent accounts of how the product build and the quality status were communicated, which limits their further generalization of this impact. Ståhl and Bosch [93] have outlined guidelines for automated integration flow, as follows:

1. Comprehensive activities. Automated testing and code analysis should be comprehensive so as to cover a meaningful number of quality assurance activities and provide a meaningful amount of information about system quality: “Construct the automated activities such that their scope affords a sufficient level of confidence in the artifacts processed by them” [93].
2. Effective communication. Communication regarding integration and the test results should be effective. “Ensure that the integration flow itself and its output are easily accessible and understandable by all its stakeholders” [93].
3. Immediacy. If the feedback flow is too slow, it may negatively impact CI efficiency and the developer’s interest in CI practice: “Make the integration flow easily and quickly accessible for the project members” [93].
4. Appropriate dimensioning. The appropriate dimensioning of the integration process and test capacity must be considered: “Adjust the capacity of the automated integration flow according to the traffic it must handle” [93].
5. Accuracy. The CI flow and system dependencies must be understood and taken into account when designing integration

and test activities: “For all activities, know well what the input is, its history and consequently, what level of confidence may be assigned to it” [93].

6. Lucidity. In complex system integrations it can be difficult to keep track of the artifact paths: “Keep the flow of changes through the integration system clear and unambiguous” [93].

Debbiche et al. [88] identified challenges relating to the developers’ mindsets, i.e., skepticism regarding the benefits of moving towards more frequent releases. In addition, Claps et al. [41] point out that organizations may have mindset challenges when adopting ARE practices. To mitigate the negative impacts, Debbiche et al. [88] suggest that higher release frequencies should not be pushed too eagerly: “Some developers feel that they lack the confidence and experience to reach desired integration frequencies. There seems to be a general consensus among developers that transitioning to CI carries risks, a period of chaos and increased pressure. Hence, the frequency of integrations and how to proceed should be done in steps in order to minimize the risk of increased pressure.” Debbiche et al. [88] suggest that the transition should leverage a bottom-up approach in terms of piloting and identifying context-specific best practices that can be shared with other teams. Debbiche et al. [88] and Claps et al. [41] both emphasize the clear communication of goals as well as instructions regarding how to proceed with the adoption of the practice. In addition, practitioners associate CD2 practice with *resistance to change* [40], *lack of motivation* [41] and *increased pressure* [41] that could decrease developer satisfaction.

According to Rissanen and Münch [62], CD can contain various technical-, procedural-, and customer-related challenges in the B2B context. In the following list, we compile 14 prerequisites for applying continuous deployment strategy. Prerequisites are extracted and synthesized from corresponding empirical studies investigating challenges in adoption of ARE practices. The list can be further used to determine project capabilities in relation to ARE.

1. All project members understand “agile development values and principles” [4,15] in software development [22,25,62,88].
2. All project members comply with “continuous integration” [11] practice in software development [20,22–25].
3. The software architecture and system modularity allow for coherent new versions to be produced at any time for CI and testing [22,62].
4. Changes in the staging and production environments are tracked, and the process is transparent to all stakeholders [62].
5. The release schedule, activities and handovers are synchronized between the internal stakeholders, i.e., developers, testers and product managers [22,25].
6. The release schedule, activities and handovers are synchronized between the external stakeholders, i.e., contractors, suppliers and customers [22,25].
7. The production system can be updated without interrupting the user [62].
8. Automated tests can assure a significant proportion of the system’s core functionality [62].
9. Third-party applications will work after pushing the changes towards the production environment [41,62].
10. Pushing the changes towards the production environment will not break the functionality with external components [62].
11. The effects on plugins and customer-specific configurations are known before pushing the changes towards the production environment [41].
12. Customer acceptance testing is integrated into the “deployment pipeline” [12,62].

13. The software branching model is clear to stakeholders, with only short-lived development branches outside the main-line/trunk [62].
14. Changes do not break the user experience, i.e. the user can fluently adopt the changes and continue using the product/service normally; otherwise, proper notifications and training should be given to the user [62].

Presumably, the strong legacy and mindset of waterfall and big bang integration practices could hinder the transition to ARE practices. Therefore, software projects in the B2B context must negotiate the details of the delivery method with the customer. As identified in several studies [22,25,62] in the B2B context, the transition to more frequent release cycles means that at least one lead customer is interested and has been incorporated into the development process. In addition, the business model, including contracts, marketing and sales strategies is also aligned with the development and release cycles [22,25]. As identified by Leppänen et al. [40], practitioners consider it beneficial to have CI capabilities and to ship software to customers *at will/on demand* or to have a fixed *date/calendar-based* deployment schedule. However, regardless of the R&D capability for automated deployment, the context and the customer-specific deployment strategy must be carefully aligned with customer and business constraints. According to Leppänen et al. [40], some practitioners consider rapid updating to be disruptive for end users. In addition, Rissanen and Münch [62], Claps et al. [41] and Karvonen et al. [22] have identified company cases where customers may be reluctant to receive new versions continuously through RR cycles. However, we could not find any empirical studies presenting first-hand evidence of this, e.g., surveys or interviews with customers to further confirm the notion of customer reluctance for CD2.

5.2. Analyzing the prevalence of the ARE practices (RQ1b)

In examining the prevalence of ARE practices, we saw there were a total of 71 primary studies that we could identify as relevant for our review. Although description of a context was often lacking, we could see that ARE practices in various software project contexts were investigated. Articles were published between the years 2005 and 2015. Considering the individual ARE practices and the Table 3 clustering of primary studies, the number of experience reports (24) and empirical studies (8) on CI topics indicates that CI practice has the most significant role among ARE practices, whereas CD, CD2 and RR have gained in popularity more recently. The awareness of modern release engineering practices and demand of empirical research is considered to increase because companies such as Mozilla are changing their release strategies towards shorter and continuous cycles [6]. Clearly, the increased market pressure and competition are also pushing companies towards practices for faster time to market.

To summarize the primary studies, it seems that fully automated deployment to the production environment (CD2) is not common practice when undertaking software projects. We could find very little evidence of actual CD2 practice usage in software projects via empirical studies. In a related synthesis, Rahman et al. [96] could only identify 19 adoptees of CD2. Interestingly, only one adoptee, Atlassian, that we also identified via our empirical study [41], was using CD2 to deploy both desktop software and websites. According to Rahman et al. [96], “Atlassian is the only adoptee that uses continuous deployment to deploy software changes for desktop software as well as websites. Rest of the 18 adoptees deploy websites using continuous deployment.” According to Claps et al. [41], CD2 was applied in a software project at Atlassian; however, it was not applied in all projects. Leppänen et al. [40] state that “Several of the companies consciously chose not to fully adopt con-

tinuous deployment. Automatic software deployment comes at the cost of relinquishing a certain level of control and human decision making, which some companies might find difficult.” However, according to [20,22,24,25], some companies seem to have the motivation and the ability to upgrade their CI capabilities towards CD2, yet they have not applied CD2 practice in their projects.

Asnawi et al. [34] investigated software professionals' perceptions and awareness of agile methodology, including CI, in the context of Malaysian software development. They pointed out that *culture* could have an effect on adoption and applicability of the practice. This is an interesting aspect that could also affect the prevalence of ARE practices in certain countries. Overall, Malaysia was considered to be behind Western countries in using agile methods [34]. Awareness of the agile methods in Malaysia was still emerging, especially in the government sector. Asnawi et al. [34] also identified several challenges associated with the adoption of agile methods, such as “people not willing to learn new things” and “lack of top management support,” etc. However, CI practice was considered to enhance feedback frequency and transparency of development progress, thus contributing to better product quality. Although the agile approach is often considered to be more established in Western companies, it seems that the challenges associated with adoption of agile methods in Malaysia are similar to the challenges associated with CD2 and IES, such as those identified in the Finnish ICT domain by Leppänen et al. [40] and Lindgren et al. [26]. Both studies identified a cultural dimension, by referring to organizational culture and resistance to change. This finding is not surprising because organizational change is often addressed as a problem, especially in business management literature [97]. However, Leppänen et al. [40] and Lindgren et al.'s [26] findings could indicate that this problem is not properly addressed in strategies for adopting ARE practices in software development projects. This challenge could be even more severe for continuous experimentation and customer involvement in development cycles, where change involves larger numbers of stakeholders and industry practices.

Deshpande and Riehle [89] analyzed CI practice usage in open-source software projects. The study indicates that open-source developers have not recently increased their use of CI practices. The assumption was that CI practice usage would have an impact on the size of the code contribution; however, the study shows that the size has remained the same. This indicates that open-source software development has not recently changed in terms of code integration practices, and that CI practice has not significantly influenced the behavior of open-source software developers. Meanwhile, Vasilescu et al. [95] investigated Travis CI service usage in GitHub projects. They discovered that most of the GitHub projects in their sample were configured to use the service, but less than half did use it. While the open-source domain is often considered to be in the forefront of agile methodologies, it is rather surprising that CI usage has not increased lately and only small portion of GitHub projects actually apply CI.

With regard to the ARE practice characteristic of frequent deliveries, it seems that in the mobile application domain, updates are provided to users rather infrequently. Statistical analysis performed by McIlroy et al. [61] on Google Play store metrics shows that only ~1% of mobile applications are updated at least once a week. This could indicate that there are some specific characteristics in the mobile domain that inhibit the usage of ARE practices.

5.3. Analyzing impacts of ARE practices on the success of software development (RQ1c)

In our analysis we identified 17 studies that investigated factors that can be applied to evaluating the success of software develop-

ment by using ARE practices (Table 4, cluster C). In this section, we elaborate the key results from these primary studies.

We identified 10 primary studies that analyzed data sets collected from Firefox browser development metrics. Consequently, the Firefox project is by far the most investigated software project in terms of understanding the impacts of rapid release. Khomh et al.'s analysis [29, 48] of Firefox browser bug metrics indicated that since moving from the 12–18-month to the 6-week release cycle, Firefox users have experienced errors earlier during software execution, i.e., the program crashes earlier than it did before the change in release cycle. Earlier crashes could be considered an indication of a somewhat deteriorated robustness of the Firefox browser. Khomh et al. [29,48] report “only a negligible difference” in the number of bugs that have been detected post-release, i.e., users do not report significantly more software bugs. Another analysis by Clark et al. [46] indicated that shifting to more frequent releases has not produced demonstrably more vulnerable (non-secure) Firefox software.

Overall, there are currently very diverging perceptions on how ARE practices could impact software quality. Based on interviews carried out by Leppänen et al. [40] and Karvonen et al. [22], some practitioners anticipate that CD2 practice could improve product quality mainly due to more frequent and automated testing. However, a transition towards IES was also considered to have a positive impact on external quality: the “main benefit of moving towards IES is to improve competitiveness and product quality, as customer feedback would increasingly impact product development” [22]. Meanwhile, Claps [41] has also reported on contradictory accounts regarding software product quality: “Quality of the software product may decrease since bugs may slip through and be deployed to customers since deployments occur more frequently.”

Ferreira and Cohen [91] analyzed the impacts on stakeholder satisfaction in the context of agile methodologies. The characteristics of agile methodologies were divided into five independent variables: *iterative development*, *continuous integration*, *collective ownership*, *test-driven design* and *feedback*. The dependent variables to measure satisfaction were *satisfaction with process* and *satisfaction with outcome*. Their study indicated that iterative development has a strong effect on stakeholder satisfaction. Consequently, CI practice was identified as important for the success of agile methodologies.

CI practice seems to stress development team communication practices by increasing team member awareness about broken builds, as stated by Downs [90]. According to Ståhl and Bosch [93], specific stakeholder information needs and viewpoints regarding the information that CI can provide should be considered in the implementation of the practice: “Ensure that the integration flow itself and its output are easily accessible and understandable by all its stakeholders” and “keep the flow of changes through the integration system clear and unambiguous. Not only does this help the status of the system to be more easily communicated, but inherent problems in its design may be more easily spotted.”

Ståhl and Bosch's [94] study also indicates that CI practice can improve developer productivity due to parallel development in the same source context. CI practice can also improve project predictability, as problems are detected early on, non-functional system testing is carried out in the early stages and integration is performed outside of the project's critical path, in contrast to traditional big bang integration at the end of the project. Meanwhile, Souza et al. [53,54] have investigated the development process by analyzing RR's effect on bug reopening (patch backouts), which may reduce productivity. Their analysis showed that “after moving to rapid releases, the bug reopening rate in Firefox increased 7%” [54].

Khomh et al.'s [29,48] statistical analysis of the Firefox browser indicates that there are several impacts on the development pro-

cess. After moving to the 6-week release cycle, bugs are fixed faster. There are fewer changes made per developer, but those changes are more invasive (affecting more files). In addition, the Firefox development focus has shifted towards continuous maintenance, i.e., the majority of changes fix errors instead of adding new features. There is no significant difference in the size and code complexity between the traditional release cycle and the 6-week release cycle. Additionally, Khomh et al. [48] address challenges in managing the RR cycle: “Overall, we identify a number of challenges that decision makers in software companies should be aware of to find the right balance between the delivery speed (release cycle) of new features and the quality of their software.” In relation to the goal of the rapid fielding of software, Bellomo et al. [63] suggest several combined software development practices to balance out development process speed and stability, such as *release planning with architecture considerations*, *prototyping/demo with quality attribute focus*, *roadmap/vision with external dependency management*, *test-driven development with quality attribute focus*, *technical debt monitoring with quality attribute focus* and *architectural change to promote stability*.

Based on the analysis of Firefox test report data before (12–18 months) and after the transition to RR cycles (6 weeks), Mäntylä et al. [13,30] report that each release cycle had a narrower scope, i.e., fewer new features were incorporated after transitioning to RR cycles. This has enabled a deeper investigation of the features and regressions with the highest risk. However, according to Mäntylä et al. [13], the open-source community has reduced its participation in RR testing, which has led Mozilla to increase its contractor resources to sustain testing.

Hemmati et al. [51] have pointed out that on large systems, it may not be feasible to rerun all tests in nightly builds. Therefore, various test-prioritization techniques must also be addressed in conjunction with RRs. According to Hemmati et al. [51], the main objectives of test-case prioritization are maximizing the test coverage and diversifying the test cases. For test prioritization, several heuristics and information sources can be used, such as *change information*, *historical fault detection information*, *dynamic and static coverage data*, *specification models/requirement documents* and *test scripts*. According to Hemmati et al.'s [51] analysis performed in the Firefox (RR) context, the *risk-driven approach* can outperform the *topic-coverage* and *text-diversity* approaches for test prioritization.

In theory, frequent releases should allow for faster releases of bug fixes. Yet, in practice, fixes can be delayed due to various reasons. Da Costa et al. [50] analyzed lead times for releasing a fix after the developer had made the changes in the development branch in conjunction with the Firefox RR approach. Out of the software fixes that were addressed in the development branch, 2% of issues were integrated into the next upcoming release. This means that 98% of the addressed Firefox issues (error fixes) were delayed by one or more releases in the RR cycle: 8% of issues were released after one release cycle, 89% of issues were released after two release cycles and 1% of issues were released after three or more releases. The median delay in releasing a Firefox issue was 42 days. However, in comparison to the traditional release approach used in ArgoUML and Eclipse projects, the delay in releasing the fix was much longer with the traditional release approach. The median delay for Eclipse projects was 112 days and for ArgoUML it was 180 days.

According to analysis performed by Souza et al. [53,54], the transition to a shorter release cycle increased the need for backouts, i.e., bug reopening and rework: “After moving to rapid releases, the bug reopening rate in Firefox increased 7%” [54]. However, an automated testing toolset helps in the early identification of issues in developers' commitment to the code and the issues can be efficiently fixed. Consequently, the current Firefox developer

community is encouraged to undertake changes more frequently. Breaking the integration repository is sometimes acceptable, but never breaking the repository could indicate that the developer is running too many tests before committing the code to the repository, as stated by Souza et al. [53].

According to Debbiche et al. [88], efficient integration and test flow in CI can deteriorate due to unstable test cases that fail regardless of the code that is being tested. However, automated tests are considered as a CI prerequisite since too high a ratio of manual testing can also diminish CI practice efficiency. According to Debbiche et al. [88], there can be some confusion regarding the dependencies between implementation and testing, e.g., developers should have a clearer understanding as to whether the test cases should be written before or after actual implementation. Preserving quality requires the appropriate dimensioning of quality assurance activities with CI flow frequency: “Pushing the integration frequency goal too eagerly could jeopardize the quality of each individual integration,” as was stated by Debbiche et al. [88].

Kerzazi and Khomh [47] analyzed factors impacting the release lead times in an industrial organization developing a web-based financial system. They identified factors in three dimensions that can be associated with software release time-to-production: 1) *technical factors*, including “merges and integration” and “testing, packaging the application;” 2) *organizational factors*, including “functional dependencies,” “design of an adapted branching structure” and “release planning;” and 3) *interactional factors*, including “coordination” and “socio-technical congruence.” Their conclusion was that the lead time of the release process is largely impacted by test activities: “86% of the release time is consumed by both manual and automated testing.” Consequently, investment in and improvement of the practices for testing can be justified, as this could potentially have a significant impact on the lead time for software release: “Release engineers need more tools and practices to implement smart automated tests in order to enhance the lead time of software releases,” as was stated by Kerzazi and Khomh [47]. In addition, in other studies—Karvonen et al. [22] and Olsson et al. [20,25]—the lack of tools and capacity for testing was mentioned as a barrier in the transition towards CD2.

6. Discussion on research implications

This section continues with the results’ interpretation and further discusses possible implications of the study. In the discussion, we also elaborated on existing research gaps and offered approaches for conducting future empirical studies on ARE practices.

6.1. Methodological approach considerations

Päivärinta et al. [10] address the need for establishing a comprehensive understanding between development practices and their impacts. This is important especially for researchers for theorizing about software development practices (e.g., how practice works in a certain context); it is also important in facilitating the organization’s learning from its own development experience. The key prerequisite for learning and software process improvement is to develop tools and methods for continuously assessing the success of the software development. The success of software development can be determined by evaluating various dimensions, such as “project schedule, cost, scope, software properties, time, market performance, and other success expectations held by the stakeholders” [10]. Päivärinta et al. states that “impacts may be desired already according to the explicit method rationale(s), or they may be unexpected, sometimes even unwanted.”

For quantification of the impacts of practice it is important to analyze and compare data collected before and after the change.

For example, analyzing software properties before and after treatment (e.g., adoption of the rapid release practice), is a feasible approach to evaluating the success of software development. Although faster release cycles allow faster introduction of product improvements and new features, transition to faster release must comply with quality standards addressed by stakeholders. Hence, for even more comprehensive understanding of the ARE impacts, it is important to investigate software properties as well as impacts on customer satisfaction, resource usage etc.

Future research on ARE practices could benefit from more extensive use of quantitative methodologies from case studies, such as those used for analyzing Firefox bug reports before and after the RR model was introduced. Moreover, combining statistical analysis with interviews could provide for a more reliable analysis of the impacts. In addition, we believe that a detailed description of the context is important to establish a baseline for meaningful analysis and comparison between software projects. Currently, empirical studies are largely based on data collected from qualitative surveys and interviews. Qualitative studies could provide insights for scoping future studies as well as providing an overview of the significance and prevalence of the practice in the software development domain. However, these findings often tend to be more vulnerable to subjective preferences and cognitive biases. To analyze the impacts on ARE practices from empirical studies, it would also be valuable to explicitly specify rationales, e.g., the motivations and goals behind why the practice was adopted and what quantitative impacts can be measured. In addition, rationales help to determine the proper metrics with which to measure the impacts. Existing case studies indicate that automated deployments are rarely adopted in organizational or project-level practice. This further challenges CD2 practice research, as opportunities for empirical studies in real software development contexts are very limited.

6.2. Stakeholder considerations

When analyzing primary studies regarding the impacts of ARE practices, we noticed that the most common view of the topic was embedded in the software supplier’s point of view. However, ARE practices also typically require the involvement of other stakeholders. We consider this as a gap in the research on ARE practices because there are many other stakeholder groups involved in the development of software systems, e.g., platform suppliers and customers.

Stakeholder satisfaction, e.g., customer satisfaction, is a common indicator of a software system’s developmental success or failure. Consequently, satisfied developers could also have a major impact on a software project’s success [98]. As explained earlier, the transition to ARE practices may have an impact on developer satisfaction. According to Rahman and Rigby [52], “Chrome switched to a shorter release cycle to avoid pressuring developers into rushing unstable code into a release... Chrome and Linux value quality over schedule and schedule over features.” While transitioning to ARE practices, developers might feel that without the traditional stabilization period, their work would not be as polished, as identified by Debbiche et al. [88]: “Some developers feel that they lack the confidence and experience to reach desired integration frequencies.” Comments like this indicate that ARE practices could be considered risky and uncomfortable if developers were obliged to commit their changes to the repository too soon and that these changes may be scheduled regardless of them having full confidence in the quality of those changes. This indicates that some developers, most likely in the early stages of CI practice adoption, would like to retain espoused development practices for traditional “big bang” integration.

6.3. ARE practices and experiment-driven development

ARE practices have often been discussed in conjunction with IES and the continuous experiment-driven approach to software development [20, 22–26]. Consequently, ARE practices may have an impact on the ability to continuously experiment with customers. As identified in interviews with practitioners [22,25,40], continuous experimentation may offer a shorter time-to-market and frequent feedback channels between customers and developers. However, empirical evidence to validate this hypothesis is still weak. In theory, feedback helps to determine functionalities that the customer wants and it could also motivate developers as they are able to feel a sense of accomplishment [40]. CD2 requires stakeholder motivation and established techniques to involve customers in providing feedback. Actions involved in the transition from traditional development to experiment-driven development have recently been discussed, especially in the context of the STH model [22,23]. The transition would require the identification of proactive (lead) customers and business ecosystem alignment [22,25]. However, in this SLR, we could not find many additional details on how this could be achieved and what the exact methods would be to involve customers in conjunction with CD2 practice. Due to scarce information on how to identify and involve customers, this topic may require further study. Interviewed software development practitioners suspect that some customers may be reluctant to receive frequent updates and test preliminary versions of the product [22,41,62]. Some business domains may also experience huge challenges when using CD2 practices. For instance, the telecom industry has been identified as a challenging domain for CD and CD2 practice in multiple studies [22,25,40]. This will further raise the question as to whether some software development domains could benefit more from using CD2 and CD practices. According to Rahman's [96] study, it seems that CD2 has so far been used almost solely for website development. Although realization of the positive impacts could also be related to the assimilation stage [16] and the actual implementation of the practice, it would be useful to investigate if some characteristics in the ecosystem and business model are more likely to result in positive impacts in terms of the usage of the CD2 approach.

6.4. ARE practices and the DevOps movement

According to Humble and Farley [12], “everybody is responsible for the delivery process.” Consequently, the DevOps movement emphasizes the same goals and principles as for CD practice: “greater collaboration between everyone involved in delivery in order to release valuable software faster and more reliably” [12]. Empirical studies [44,45] emphasize DevOps as a new phenomenon that has as its goal the bridging of the gap between development and operations. However, existing empirical studies still focus mostly on defining what the DevOps concept means and on the investigation of impediments to the adoption of DevOps [44] based on practitioner interviews. In a similar vein to research on CD and CD2, it seems that the number of software companies implementing DevOps is still rather small and opportunities for empirical research in companies are limited. To summarize the relationship between the DevOps movement and ARE practices, it seems that the goals are mostly congruent and DevOps as a research topic is highly relevant for release engineering and the continuous software engineering research domains.

6.5. Rationales for using ARE practices

Rationales for applying ARE practices are closely related to impacts; however, the actual realization and impacts can differ from what the intended rationale was for using the practice. Although

our research goal was to identify the impacts, we consider it important to also briefly discuss the rationales for ARE practices. Lepänen et al. [40] state that context-specific motivations for CD2 deserve more research and experience sharing. We were able to identify several rationales for using ARE practices in software projects such as a *strategic transition to an experiment-driven development paradigm* [20,22,23,25], *customer demand for more frequent releases* [24], *unification of development practices* [34], *good experiences from a previous project* [34], *competitors' release model change* [30], *avoiding rushing unstable code into a release* [52], *the complexity of product integration* [88] and *accumulating a technical testing debt* [79]. This list is not by any means an exhaustive analysis of the rationales for ARE practices; hence, a thorough analysis would require a more comprehensive study. In the practitioner's literature, ARE practices are often considered to have rather direct impacts on operational costs, productivity and product quality. These benefits, however, are rarely measured or validated in empirical studies. To summarize, ARE practices have an impact on software quality, and except for analyses undertaken mostly for the Firefox product, it seems that impacts are rarely quantified at all in empirical studies. Consequently, it is not comprehensively understood how the practice may have an impact on, for example, software quality. Instead, there could even be contradictory practitioner beliefs regarding how the practice might have an impact on quality. Test automation could intuitively improve software quality since tests can be executed repeatedly; however, more frequent releases still pose many challenges in terms of testing each release comprehensively. In relation to ARE practices, testing activities seem to play an exceptionally important role. Whereas in traditional releases, i.e., ~12-month cycles, it is possible to plan and run the whole test suite for each release, for rapid weekly releases it becomes more important to plan and run smaller test suites and prioritize test cases.

6.6. Development process and ecosystem considerations

It seems that the utility, speed and extent of applying ARE practices is largely ruled by the boundaries and consent of the business and software ecosystem processes. As stated earlier, the utility of ARE practices can be affected by domain suitability and support. This also clearly addresses planning of methods and approaches for conducting empirical studies. The optimal release cycle, i.e., the update frequency, could be dependent on the software ecosystem deployment tools, practices and policies. For example, as stated by Facebook's release engineering manager, Chuck Rossi, as referred to by Adams et al. [2], “Mobile deployments are more challenging than web deployments because we don't own the ecosystem, so we can't do all the things that we would do normally.” According to Rossi [99], the transition to RRs requires continuous improvements in the development process and organizational structures. Rossi emphasizes how release engineering practices have to manage three dimensions: 1) schedule, 2) quality and 3) features. In Facebook's fixed date-based release model, the schedule and quality are always fixed. Facebook's release engineering policy is that features that do not pass release quality criteria will be postponed to later releases. Facebook releases web applications twice per day and mobile applications once per week (on the iOS and Android platform). According to Rossi, as referred to by Adams et al. [2], “Continuous deployment works for small teams, with 20 to 30 changes per day.” Facebook prefers to manually “cherry-pick” changes from the development branches to be pushed into the production environment, i.e., they use various manual and automated quality assurance methods as they gather data regarding the effects of each change to control the quality of the production environment by selecting changes that are proven to work and will not break existing functionality. Firefox (a web browser on multiple

PC and mobile platforms) [28], Facebook (web and mobile applications) [99] and Chrome (a web browser application on PCs and mobile platforms) [27] have fixed date-based deployments with multiple release channels; they all employ manual code reviews and automated quality checks before they push changes into the production environment (web and mobile applications).

7. Limitations of the study and future research ideas

In literature reviews, the main validity threats and limitations typically lie in the search, selection and analysis of the primary studies. As explained earlier, we performed search, selection and quality evaluations with two authors and cross-checked each decision to avoid the subjective selection of primary studies to be used in the analysis. Furthermore, the analysis and findings of the paper were reviewed by all of the authors. Our search was carried out using the specific concepts of *continuous delivery*, *continuous deployment*, *rapid release* and *continuous integration*. Due to our search method, it is possible that we may have missed some empirical studies for ARE practices that might have been found by using different variations in the searched-for concepts or by adding more search terms, such as “agile” or “extreme programming,” etc. Nevertheless, we believe that the number of included studies provide enough data to synthesize this topic in a meaningful way.

The identified impacts are extracted from the results of empirical studies; they should be considered to be context-sensitive and not directly generalizable to the entire software development domain. Also, aggregating practices and impacts between very different research contexts addresses issues with construct validity. Consequently, although we have some information regarding the context, practice and impacts, the level of detail in primary studies clearly is not adequate, for example, for making meaningful comparisons between explicit impacts on issues such as post-deployment bugs.

Due to the number of empirical studies and to limit the paper's length, we made a decision to pay less attention to analyzing experience reports. Therefore, the analysis presented in Section 5 is based on evidence extracted from empirical studies. We acknowledge that experience reports could probably provide relevant insights for our analysis. However, the validity of claims regarding impacts presented in experience reports is more questionable due to missing descriptions of the research settings, design and methods. Nevertheless, experience reports most likely could provide more answers to questions on how to implement ARE practices.

When synthesizing results from multiple studies, it is important to also take note of the possible tendency to report positive results over negative results when considering the usage of ARE practices. This may expose a validity threat regarding both experience reports and empirical studies. Specifically, exposing a company name to negative publicity could block the reporting of any negative experiences and impacts when using ARE practices.

We believe that the above analysis can provide a sufficiently comprehensive picture of the current state of impacts identified in empirical studies. This also helps further in identifying knowledge gaps that could be filled in with future studies. As for future studies, we plan to extend the analysis of the impacts of ARE practices presented in this paper with industry case studies. Specifically, we want to empirically validate and further develop the checklist for analyzing company and ecosystem readiness and capabilities for CD2 and CD.

8. Conclusion

This paper presented a synthesis of the impacts of ARE practices by analyzing empirical studies that had investigated continu-

ous integration, continuous delivery, continuous deployment, and rapid release. One of our main research objectives was to understand how ARE practices have been investigated in empirical studies (RQ2). Our results indicated that 33 out of 71 primary studies were experience reports that had neither an explicit research method nor a data collection approach specified. Twenty-three out of 38 empirical studies applied qualitative methods, such as interviews and surveys, among practitioners. Twelve empirical studies applied quantitative methods, such as investigating software repositories and work products (e.g., log files and bug reports). Only three empirical studies combined these research approaches. As illustrated in Fig. 2, we identified four main research approaches that were used in the primary studies: 1) impacts associated with the adoption of the practice; 2) impacts associated with the prevalence and significance of the practice in the software development domain; 3) impacts associated with the success of the software development; and 4) experience reports that were used to provide insights into how the practice can be implemented. The identified impacts from the primary studies were further discussed in terms of impacts associated with adoption (RQ1a), prevalence (RQ1b), and success of software development (RQ1c).

Existing empirical evidence, especially from the Firefox context before and after the adoption of RR cycles, indicates that there can be many impacts, both on the development process and on the software quality when transitioning to RR. While these findings are important, it is too early to generalize ARE practice impacts on the software development domain. However, these findings suggest that companies should be cautious when transitioning to faster release cycles or, at least, they should plan the transition well.

Based on analyzed empirical studies, continuous and fully automated delivery to the customer is not a prevailing practice in software development projects. While practitioners and researchers often anticipate many benefits from using ARE practices, empirical studies also indicate a degree of skepticism and ambiguity in practitioners' perceptions regarding the anticipated positive impacts of ARE practices in relation to software and development process quality as well as stakeholder satisfaction. Practitioners are also doubtful regarding customer motivation and willingness in terms of having faster release cycles. A broader understanding of ARE practices from different stakeholder points of view, such as customer perceptions regarding ARE practices, clearly deserve more research.

The presented synthesis of the impacts of ARE practices provides a summary of empirical studies conducted in various software development contexts. Earlier studies suggest that practice in theory and actual implementation could vary from case to case [17]. Consequently, meaningful comparisons of the impacts between software projects in relation to ARE practices would require a detailed description of the software development context and implementation of the practice. Thus, future studies should pay more attention to describing the software development context and elaborating on how the practice was implemented.

The checklist for analyzing the project's capabilities for CD was compiled from primary studies. In addition, ARE practice relationships to emerging research topics in software engineering, experiment-driven development and DevOps were also elaborated. To summarize the current empirical evidence, ARE practices can pave the way toward the early detection of errors, better communication within and between development teams and a higher level of transparency and efficiency in the development process. Moreover, ARE practices are also a relevant topic in the context of the DevOps community and when transitioning to the experiment-driven development paradigm.

Acknowledgements

This work was supported by TEKES as part of the Need for Speed Project (<http://www.n4s.fi/>) of DIMECC (Digital, Internet, Materials & Engineering Co-Creation).

Supplementary materials

Supplementary material associated with this article can be found, in the online version, at [doi:10.1016/j.infsof.2017.01.009](https://doi.org/10.1016/j.infsof.2017.01.009).

References

- [1] G. Ruhe, Product Release Planning Methods, Tools and Applications, Auerbach Publications, Taylor and Francis Group, LLC, 2010.
- [2] B. Adams, S. Bellomo, C. Bird, T. Marshall-Keim, F. Khomh, K. Moir, The practice and future of release engineering: a roundtable with three release engineers, *IEEE Softw.* 32 (2015) 42–49.
- [3] B. Adams, S. McIntosh, Modern release engineering in a nutshell: why researchers should care, in: *Proc. of the International Conference on Software Analysis, Evolution, and Reengineering (SANER)*, 2016, pp. 78–90.
- [4] K. Beck, C. Andres, *Extreme Programming Explained: Embrace Change*, 2nd Edition (The XP Series) 2nd Edition, Addison-Wesley, 2004.
- [5] M. Poppendieck, M.A. Cusumano, Lean software development: a tutorial, *IEEE Softw.* 29 (2012) 26–32.
- [6] B. Adams, C. Bird, F. Khomh, K. Moir, 1st International workshop on release engineering (RELENG 2013), in: *2013 35th International Conference on Software Engineering (ICSE)*, IEEE, 2013, pp. 1545–1546.
- [7] J. Bosch, Continuous software engineering: an introduction, *Contin. Softw. Eng.* (2014).
- [8] B. Fitzgerald, K.J. Stol, Continuous software engineering: a roadmap and agenda, *J. Syst. Softw.* (2015).
- [9] J.D. Womack, D.T. Jones, *Lean Thinking: Banish Waste and Create Wealth in Your Corporation*, Productivity Press, 2003 Revised and Updated.
- [10] T. Päiväranta, K. Smolander, Theorizing about software development practices, *Sci. Comput. Program.* 101 (2015) 124–135.
- [11] P. Duvall, S. Matyas, A. Glover, *Continuous Integration: Improving Software Quality and Reducing Risk*, Pearson Education, 2007.
- [12] J. Humble, D. Farley, *Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation*, 2010.
- [13] M.V. Mäntylä, B. Adams, F. Khomh, E. Engström, K. Petersen, On rapid releases and software testing: a case study and a semi-systematic literature review, *Empir. Softw. Eng.* 20 (2014) 1384–1425.
- [14] B. Kitchenham, in: *Procedures for Performing Systematic Reviews*, 33, Keele Univ., Keele, UK, 2004, p. 28.
- [15] Beck, K., Beedle, M., Van Bennekum, A., Cockburn, A., Cunningham, W., Fowler, M., Grenning, J., Highsmith, J., Hunt, A., Jeffries, R., Kern, J., Marick, B., Martin, R.C., Mellor, S., Schwaber, K., Sutherland, J., Thomas, D.: *Agile Manifesto*, <http://agilemanifesto.org/>.
- [16] A. Eck, F. Uebernickel, W. Brenner, Fit for continuous integration: how organizations assimilate an agile practice, 20th Americas Conference on Information Systems, AMCIS 2014, Association for Information Systems, 2014.
- [17] D. Ståhl, J. Bosch, Modeling continuous integration practice differences in industry software development, *J. Syst. Softw.* 87 (2014) 48–59.
- [18] Humble, J.: *Continuous Delivery vs Continuous Deployment*, <http://continuousdelivery.com/2010/08/continuous-delivery-vs-continuous-deployment/>.
- [19] P. Rodríguez, A. Haghighatkhah, L.E. Lwakatare, S. Teppola, T. Suomalainen, J. Eskeli, T. Karvonen, P. Kuvaja, J.M. Verner, M. Oivo, Continuous deployment of software intensive products and services: a systematic mapping study, *J. Syst. Softw.* (2016).
- [20] H.H. Olsson, H. Alahyari, J. Bosch, Climbing the “Stairway to heaven” – A multiple-case study exploring barriers in the transition from agile development towards continuous deployment of software, in: *Software Engineering and Advanced Applications (SEAA)*, 2012 38th EUROMICRO Conference on IEEE, 2012, pp. 392–399.
- [21] J. Bosch, *Building Products as Innovation Experiment Systems*, Springer, Berlin Heidelberg, Berlin, Heidelberg, 2012.
- [22] T. Karvonen, L.E. Lwakatare, T. Sauvola, J. Bosch, H.H. Olsson, P. Kuvaja, M. Oivo, Hitting the target: practices for moving toward innovation experiment systems, in: *Proceedings of the Software Business 6th International Conference, ICSOB 2015, Braga, Portugal, 210, 2015*, pp. 117–131.
- [23] H.H. Olsson, J. Bosch, Climbing the stairway to heaven: evolving from agile development to continuous deployment of software, in: *Continuous Software Engineering*, 2014, pp. 15–27.
- [24] H.H. Olsson, J. Bosch, Towards agile and beyond: An empirical account on the challenges involved when advancing software development practices, in: *Agile Processes in Software Engineering and Extreme Programming*, 179, LNBI, 2014, pp. 327–335.
- [25] H.H. Olsson, J. Bosch, H. Alahyari, Towards R&D as innovation experiment systems: a framework for moving beyond agile software development, in: *Artificial Intelligence and Applications / 794: Modelling, Identification and Control / 795: Parallel and Distributed Computing and Networks / 796: Software Engineering / 792: Web-based Education*, 2013, pp. 798–805.
- [26] E. Lindgren, J. Münch, *Software Development as an Experiment System: A Qualitative Survey on the State of the Practice*, Springer International Publishing, Cham, 2015.
- [27] Chromium: Chrome Release Channels, <https://www.chromium.org/getting-involved/dev-channel>.
- [28] Mozilla: Release Management/Release Process, <https://wiki.mozilla.org/RapidRelease>.
- [29] F. Khomh, T. Dhaliwal, Y. Zou, B. Adams, Do faster releases improve software quality? An empirical case study of Mozilla Firefox, in: *IEEE Int. Work. Conf. Min. Softw. Repos.*, 2012, pp. 179–188.
- [30] M.V. Mäntylä, F. Khomh, B. Adams, E. Engström, K. Petersen, On rapid releases and software testing, in: *IEEE Int. Conf. Softw. Maintenance, ICSM*, 2013, pp. 20–29.
- [31] B. Kitchenham, P. Brereton, A systematic review of systematic review process research in software engineering, *Inf. Softw. Technol.* 55 (2013) 2049–2075.
- [32] QRS International, NVivo, 2016 <http://www.qsrinternational.com/>.
- [33] M. Ivarsson, T. Gorschek, A method for evaluating rigor and industrial relevance of technology evaluations, *Empir. Softw. Eng.* 16 (2010) 365–395.
- [34] A.L. Asnawi, A.M. Gravell, G.B. Wills, Emergence of agile methods: perceptions from software practitioners in Malaysia, in: *Proc. - Agil. India 2012, Agil. 2012*, 2012, pp. 30–39.
- [35] P. Sfetsos, L. Angelis, I. Stamelos, Investigating the extreme programming system – An empirical study, *Empir. Softw. Eng.* 11 (2006) 269–301.
- [36] P. Tingling, A. Saeed, Extreme programming in action: a longitudinal case study, in: *Human-Computer Interaction. Interaction Design and Usability*, 2007, pp. 242–251.
- [37] J.M. Bass, Influences on agile practice tailoring in enterprise software development, in: *Proc. - Agil. India 2012, Agil. 2012*, 2012, pp. 1–9.
- [38] C. Escoffier, O. Günalp, P. Lalanda, *Requirements to Pervasive System Continuous Deployment*, Springer International Publishing, Cham, 2014.
- [39] A. Claassen, L. Boekhorst, *Agile processes, Software Engineering, and Extreme Programming*, Springer International Publishing, Cham, 2015.
- [40] M. Leppänen, S. Mäkinen, M. Pagels, V.-P. Eloranta, J. Itkonen, M.V. Mäntylä, T. Männistö, The highways and country roads to continuous deployment, *IEEE Softw.* 32 (2015) 64–72.
- [41] G.G. Claps, R. Berntsson Svensson, A. Aurum, On the journey to continuous deployment: technical and social challenges along the way, *Inf. Softw. Technol.* 57 (2015) 21–31.
- [42] A. Schaefer, M. Reichenbach, D. Fey, *Continuous Integration and Automation for DevOps*, Springer, Netherlands, Dordrecht, 2013.
- [43] D. Cukier, DevOps patterns to scale web applications using cloud services, in: *Proceedings of the 2013 Companion Publication for Conference on Systems, Programming, & Applications: Software for Humanity - SPLASH '13*, 2013, pp. 143–152.
- [44] J. Smeds, K. Nybom, I. Porres, *DevOps: A Definition and Perceived Adoption Impediments*, Springer International Publishing, Cham, 2015.
- [45] L.E. Lwakatare, P. Kuvaja, M. Oivo, *Dimensions of DevOps*, Springer International Publishing, Cham, 2015.
- [46] S. Clark, M. Collis, M. Blaze, J.M. Smith, Moving targets: security and rapid-release in Firefox, in: *Proceedings of the 2014 ACM SIGSAC Conference on Computer and Communications Security - CCS '14*, ACM Press, New York, New York, USA, 2014, pp. 1256–1266.
- [47] N. Kerzazi, F. Khomh, Factors impacting rapid releases: an industrial case study, in: *Proceedings of the 8th ACM/IEEE International Symposium on Empirical Software Engineering and Measurement - ESEM '14*, ACM Press, New York, New York, USA, 2014, pp. 1–8.
- [48] F. Khomh, B. Adams, T. Dhaliwal, Y. Zou, Understanding the impact of rapid releases on software quality: the case of Firefox, *Empir. Softw. Eng.* 20 (2014) 336–373.
- [49] O. Baysal, O. Kononenko, R. Holmes, M.W. Godfrey, The secret life of patches: a Firefox case study, in: *2012 19th Working Conference on Reverse Engineering*, 2012, pp. 447–455.
- [50] D.A. da Costa, S.L. Abebe, S. McIntosh, U. Kulesza, A.E. Hassan, An empirical study of delays in the integration of addressed issues, in: *2014 IEEE International Conference on Software Maintenance and Evolution*, 2014, pp. 281–290.
- [51] H. Hemmati, Z. Fang, M.V. Mantyla, Prioritizing manual test cases in traditional and rapid release environments, in: *2015 IEEE 8th International Conference on Software Testing, Verification and Validation (ICST)*, IEEE, 2015, pp. 1–10.
- [52] M.T. Rahman, P.C. Rigby, Release stabilization on Linux and Chrome, *IEEE Softw.* 32 (2015) 81–88.
- [53] R. Souza, C. Chavez, R.A. Bittencourt, Rapid releases and patch backouts: a software analytics approach, *IEEE Softw.* 32 (2015) 89–96.
- [54] R. Souza, C. Chavez, R.A. Bittencourt, Do rapid releases affect bug reopening? A case study of Firefox, in: *2014 Brazilian Symposium on Software Engineering*, IEEE, 2014, pp. 31–40.
- [55] J. Gmeiner, R. Ramler, J. Haslinger, Automated testing in the continuous delivery pipeline: a case study of an online company, in: *Software Testing, Verification and Validation Workshops (ICSTW)*, 2015 IEEE Eighth International Conference on, 2015, pp. 1–6.
- [56] P. Agarwal, Continuous SCRUM: agile management of SAAS products, in: *Proceedings of the 4th India Software Engineering Conference*, 2011, pp. 51–60.
- [57] S. Neely, S. Stolt, Continuous delivery? Easy! Just change everything (well, maybe it is not that easy), in: *Proc. - Agil. 2013, 2013*, pp. 121–128.

- [58] L. Chen, Continuous delivery: huge benefits, but challenges too, *IEEE Softw.* (2015) 1.
- [59] S. Cohan, Successful customer collaboration resulting in the right product for the end user, in: *Proc. - Agil. 2008 Conf.*, 2008, pp. 284–288.
- [60] S. Bellomo, N. Ernst, R. Nord, R. Kazman, Toward design decisions to enable deployability: empirical study of three projects reaching for the continuous delivery holy grail, in: *2014 44th Annual IEEE/IFIP International Conference on Dependable Systems and Networks*, 2014, pp. 702–707.
- [61] S. McIlroy, N. Ali, A.E. Hassan, Fresh apps: an empirical study of frequently-updated mobile apps in the Google play store, *Empir. Softw. Eng.* (2015).
- [62] O. Rissanen, J. Münch, Transitioning Towards Continuous Delivery in the B2B Domain: A Case Study, Springer International Publishing, Cham, 2015.
- [63] S. Bellomo, R.L. Nord, I. Ozkaya, A study of enabling factors for rapid fielding combined practices to balance speed and stability, in: *Proc. - Int. Conf. Softw. Eng.*, 2013, pp. 982–991.
- [64] C.P. Baker, Supporting “agile” development with a continuous integration system, in: *Twenty-Third Annu. Pacific Northwest Softw. Qual. Conf. (PNSQC 2005)*, *Proc.*, 2005, pp. 445–457.
- [65] C. Basarke, C. Berger, B. Rumpe, Software & systems engineering process and tools for the development of autonomous driving intelligence, *J. Aerosp. Comput. Inf. Commun.* 4 (2007) 1158–1174.
- [66] R.M. Betz, R.C. Walker, Streamlining development of a multimillion-line computational chemistry code, *Comput. Sci. Eng.* 16 (2014) 10–17.
- [67] M.S. Botsali, A. Tumay, R. Acun, K. Dincer, Integrated working environment as a tool to increase efficiency and productivity, in: *Proc. 3RD Int. Conf. Inf. Manag. Eval.*, 2012, pp. 347–350.
- [68] G. Brooks, Team pace - keeping build times down, in: *Agile 2008 Conference*, 2008, pp. 294–297.
- [69] F. Cannizzo, R. Clutton, R. Ramesh, Pushing the boundaries of testing and continuous integration, in: *Agil. 2008 Conf.*, 2008, pp. 501–505.
- [70] D. Goodman, M. Elbaz, “It’s not the pants, it’s the people in the pants” learnings from the gap agile transformation what worked, how we did it, and what still puzzles us, in: *Agile 2008 Conference*, *IEEE*, 2008, pp. 112–115.
- [71] J. Ivanovs, K. Rauhvargers, Handling server-side software versioning: the “smart technology” approach, *Front. Artif. Intell.* 249 (2013) 303–316.
- [72] A.H. Khan, A.M. Memon, Enhancing Testing Technologies for Globalization of Software Engineering and Productivity, 2009.
- [73] M. Kulas, J.L. Borelli, W. Gässler, D. Peter, S. Rabien, G. Orban de Xivry, L. Busoni, M. Bonaglia, T. Mazzoni, G. Rahmer, Practical experience with test-driven development during commissioning of the multi-star AO system ARGOS, in: G. Chiozzi, N.M. Radziwill (Eds.), *Proceedings of SPIE - The International Society for Optical Engineering*, SPIE, 2014, p. 91520D.
- [74] F.J. Lacoste, Killing the gatekeeper: introducing a continuous integration system, in: *Proc. - 2009 Agil. Conf. Agil.* 2009, 2009, pp. 387–392.
- [75] J. Lantz, U. Eliasson, Scaling agile mechatronics: an industrial case study, in: *Continuous Software Engineering*, Springer International Publishing, 2014, pp. 211–222.
- [76] J. Lu, Z. Yang, J. Qian, Implementation of continuous integration and automated testing in software development of smart grid scheduling support system, in: *2014 International Conference on Power System Technology*, 2014, pp. 2441–2446.
- [77] J. McCarty, J. Morris, NMotion: a continuous integration system for NASA software, *28th Space Simulation Conference - Extreme Environments: Pushing the Boundaries*, 2014.
- [78] A. Miller, A hundred days of continuous integration, in: *Proc. - Agil. 2008 Conf.*, 2008, pp. 289–293.
- [79] S. Stolberg, Enabling agile testing through continuous integration, in: *Proc. - 2009 Agil. Conf. Agil.* 2009, 2009, pp. 369–374.
- [80] T. Su, J. Lyle, A. Atzeni, S. Faily, H. Virji, C. Ntanos, C. Botsikas, Continuous integration for web-based software infrastructures: lessons learned on the webinos project, in: *Hardware and Software: Verification and Testing*, 8244, LNCS, 2013, pp. 145–150.
- [81] J. Sutherland, R. Frohman, Hitting the wall: What to do when high performing scrum teams overwhelm operations and infrastructure, in: *Proc. Annu. Hawaii Int. Conf. Syst. Sci.*, 2011, pp. 1–6.
- [82] H. Søvik, M. Forfang, Tech challenges in a large-scale agile project, in: *Agile Processes in Software Engineering and Extreme Programming*, 48, LNBP, 2010, pp. 353–361.
- [83] M. Yap, Follow the sun: distributed extreme programming development, in: *Proceedings of the Agile Development Conference*, 2005, pp. 218–224.
- [84] H.M. Yuksel, E. Tuzun, E. Gelirli, E. Biyikli, B. Baykal, Using continuous integration and automated test techniques for a robust C4ISR system, in: *2009 24th International Symposium on Computer and Information Sciences*, 2009, pp. 743–748.
- [85] Y.V. Zaytsev, A. Morrison, Increasing quality and managing complexity in neuroinformatics software development with continuous integration, *Front. Neuroinform.* 6 (2012) 31.
- [86] E. Kim, S. Ryoo, Agile adoption story from NHN, in: *Computer Software and Applications Conference (COMPSAC)*, 2012 IEEE 36th Annual., 2012, pp. 476–481.
- [87] R.M. Betz, R.C. Walker, Implementing continuous integration software in an established computational chemistry software package, in: *2013 5th Int. Work. Softw. Eng. Comput. Sci. Eng. SE-CSE 2013 - Proc.*, 2013, pp. 68–74.
- [88] A. Debiche, M. Dienér, R.B. Svensson, Challenges when adopting continuous integration: a case study, in: *Product-Focused Software Process Improvement*, 8892, 2014, pp. 17–32.
- [89] A. Deshpande, D. Riehle, Continuous integration in open source software development, in: *Open Source Development, Communities and Quality*, 2008, pp. 273–280.
- [90] J. Downs, J. Hosking, B. Plimmer, Status communication in agile software teams: aA case study, in: *Proc. - 5th Int. Conf. Softw. Eng. Adv. ICSEA 2010*, 2010, pp. 82–87.
- [91] C. Ferreira, J. Cohen, Agile systems development and stakeholder satisfaction: a South African empirical study, in: *Proc. 2008 Annu. Res. Conf. South African Inst. Comput. Sci. Inf. Technol. IT Res. Dev. Ctries. Rid. wave Technol.* 6–8 Oct., 2008, pp. 48–55.
- [92] E. Knauss, A. Andersson, M. Rybacki, E. Israelsson, Research preview: Supporting requirements Feedback Flows in Iterative System Development, Springer International Publishing, Cham, 2015.
- [93] D. Ståhl, J. Bosch, Automated software integration flows in industry: a multiple-case study, in: *Companion Proceedings of the 36th International Conference on Software Engineering - ICSE Companion 2014*, 2014, pp. 54–63.
- [94] D. Ståhl, J. Bosch, Experienced benefits of continuous integration in industry software product development: a case study, in: *Artificial Intelligence and Applications / 794: Modelling, Identification and Control / 795: Parallel and Distributed Computing and Networks / 796: Software Engineering / 792: Web-based Education*, 2013, pp. 736–743.
- [95] B. Vasilescu, S.van Schuylenburg, J. Wulms, A. Serebrenik, M.G.J. van den Brand, Continuous integration in a social-coding world: empirical evidence from github, in: *2014 IEEE International Conference on Software Maintenance and Evolution*, 2014, pp. 401–405.
- [96] A.A.U. Rahman, E. Helms, L. Williams, C. Parnin, Synthesizing continuous deployment practices used in software development, in: *Proceedings - 2015 Agile Conference, Agile 2015*, IEEE, 2015, pp. 1–10.
- [97] J.P. Kotter, *Sense of Urgency*, Harvard Business Press, 2008.
- [98] J.D. Procaccino, J.M. Verner, S.J. Lorenzet, Defining and contributing to software development success, *Commun. ACM* 49 (2006) 79–83.
- [99] Facebook: *Hacker Way: Releasing and Optimizing Mobile Apps for the World*, <https://www.youtube.com/watch?v=mOyoTUEtmSM>.